

OCA I3C Controller — Functional Specification

OCA I3C Controller — Specification (External Contract)

1. Role
2. Theory of operation
 - 2.1 Multi-instance aperture
 - 2.2 The PIO programming model
 - 2.3 In-Band Interrupts
 - 2.4 Functional abstraction boundary
3. Register map (per instance, window 0x000..0x4FF)
4. HC_CONTROL (0x04)
5. RESET_CONTROL (0x10)
6. Interrupts
 - 6.1 PIO_INTR_STATUS (0xA0)
 - 6.2 INTR_STATUS (0x20) — HC level (all latched, W1C)
 - 6.3 Aggregation
7. Command descriptor (model 64-bit layout)
8. Response descriptor (32-bit) — RESPONSE_PORT
9. IBI status descriptor (32-bit) — IBI_PORT
10. Transaction semantics (observable)
11. TLM-2.0 interface
12. Documented simplifications (vs RTL)

OCA I3C Controller — Specification (External Contract)

Status: Modelled · **Style:** SystemC / TLM-2.0 Loosely-Timed (LT) · **CCI:** 1.0

This document is the **external, RTL-traceable contract** of the OCA I3C Controller SystemC model. It describes *what* the model does (register map, observable behaviour, error responses) without prescribing *how* — see `02_I3C_CONTROLLER_LowLevel_Design.md` for the internal design.

1. Role

The OCA I3C Controller is the I3C/I2C communication peripheral of the System Management Controller (SMC). It is a MIPI **I3C Basic v1.0/v1.1.1 + HCI v1.2** compliant controller (CHIPS-Alliance i3c-core fork with OCA enhancements) supporting active-controller, secondary-controller, and target operation with I2C backward compatibility. In the SMC it is instantiated as a **multi-instance wrapper** (`i3ccore_wrapper`) exposing **six** independent I3C instances behind a single AXI4-Lite slave. It:

1. Provides each instance with an **I3CCSR** 32-bit register block (HCI v1.2 layout) for firmware configuration, queue access, and status.
2. Manages descriptor-based **HCI queues** — command, response, TX data, RX data, and IBI — with threshold-based interrupts.
3. Drives I3C bus transactions (private write/read, CCC, IBI handling) and reports completion through response descriptors.
4. Raises a **per-instance interrupt** aggregating the enabled HC-level and PIO-level status sources.

RTL references: `i3ccore_wrapper.sv`, `i3c_wrapper.sv`, `i3c.sv`, `i3ccore_wrap_pkg.sv`, `oca_i3c_wrap.rdl`, `architecture.adoc`, `memmap.adoc`, `smc_peripherals.sv`, `smc_config_pkg.sv`, `smc_top.rdl`.

2. Theory of operation

2.1 Multi-instance aperture

The wrapper packs `NUM_I3C` (default 6) independent instances at a fixed `INSTANCE_SPACING` of `0x500` bytes. An AXI-Lite access is routed to instance `floor(offset / 0x500)`, and the per-instance register at `offset mod 0x500`. Each instance has an 11-bit register window whose live map occupies `0x000..0x4FF`. The SMC places the aperture at `OCA_I3C_WRAP_0_REG_MAP_BASE_ADDR = 0xC000_5000`; the model exposes offsets relative to its own aperture base.

2.2 The PIO programming model

Firmware drives the controller in **PIO mode** (`HC_CONTROL.MODE_SELECTOR` reads as 1; DMA mode is not implemented in OCA). The flow is:

1. Populate the **Device Address Table** (DAT) so each `dev_index` maps to a target's 7-bit dynamic address.
2. Configure thresholds (`QUEUE_THLD_CTRL`, `DATA_BUFFER_THLD_CTRL`) and interrupt masks (`INTR_*`, `PIO_INTR_*`).
3. Set `HC_CONTROL.BUS_ENABLE`.

4. For each transfer, push a 64-bit **command descriptor** to `COMMAND_PORT` (two 32-bit writes) and — for writes — the payload DWORDs to `XFER_DATA_PORT`.
5. The controller executes the transaction, pushes a 32-bit **response descriptor** to the response queue, and raises `RESP_READY_STAT`.
6. Firmware reads `RESPONSE_PORT`, and (for reads) drains the RX FIFO via `XFER_DATA_PORT`.

2.3 In-Band Interrupts

Targets asynchronously notify the controller via IBIs. An accepted IBI yields an **IBI status descriptor** (source address + payload length) followed by the payload DWORDs in the IBI queue, and raises `IBI_STATUS_THLD_STAT`. Firmware reads them through `IBI_PORT`.

2.4 Functional abstraction boundary

This is an LT/firmware model. The **register and queue behaviour, response semantics, and interrupt aggregation are reproduced faithfully**; the bit-level **SCL/SDA waveform, OD/PP timing parameters, CDC synchronisers, HDR modes, and the external DAT/DCT memory** are abstracted (see §10). Data movement on the wire is modelled by an attachable **bus model** callback that emulates targets.

3. Register map (per instance, window 0x000..0x4FF)

All registers are 32-bit and accept only 32-bit, naturally-aligned access.

Offset	Name	SW	Reset	Description
0x00	HCI_VERSION	R	0x0000_0120	HCI v1.2 version.
0x04	HC_CONTROL	RW	0x0000_0040	Host-controller control (see §4).
0x08	CONTROLLER_DEVICE_ADDR	RW	0x0000_0000	Controller dynamic address + valid.
0x0C	HC_CAPABILITIES	R	(cfg)	Feature flags.
0x10	RESET_CONTROL	RW	0x0000_0000	Self-clearing soft/queue resets (see §5).
0x14	PRESENT_STATE	R	(hw)	bit2 AC_CURRENT_OWn .
0x20	INTR_STATUS	RW1C	0x0000_0000	HC-level error/status (see §6).
0x24	INTR_STATUS_ENABLE	RW	0x0000_0000	HC status mask.
0x28	INTR_SIGNAL_ENABLE	RW	0x0000_0000	HC signal routing.
0x2C	INTR_FORCE	W	—	Write-1 sets the matching INTR_STATUS bit.
0x30	DAT_SECTION_OFFSET	R	(cfg)	DAT offset/size/entry.
0x34	DCT_SECTION_OFFSET	RW	(cfg)	DCT offset/size/entry; ENTDAAs index RW.
0x3C	PIO_SECTION_OFFSET	R	0x0000_0080	PIO base pointer.
0x80	COMMAND_PORT	W	—	64-bit command descriptor (two writes).
0x84	RESPONSE_PORT	R	—	Pop a 32-bit response descriptor.
0x88	XFER_DATA_PORT	RW	—	Write = TX push; read = RX pop.
0x8C	IBI_PORT	R	—	Pop IBI status / data.
0x90	QUEUE_THLD_CTRL	RW	0x0101_0101	Command/response/IBI thresholds.
0x94	DATA_BUFFER_THLD_CTRL	RW	0x0101_0101	TX/RX buffer thresholds (powers of 2).
0x98	QUEUE_SIZE	R	(cfg)	CR/IBI/RX/TX queue sizes.
0x9C	ALT_QUEUE_SIZE	R	0x0000_0000	Alternate/extended queue size.
0xA0	PIO_INTR_STATUS	RW1C	0x0000_0000	PIO threshold/transfer status (see §6).
0xA4	PIO_INTR_STATUS_ENABLE	RW	0x0000_0000	PIO status mask.
0xA8	PIO_INTR_SIGNAL_ENABLE	RW	0x0000_0000	PIO signal routing.
0xAC	PIO_CONTROL	RW	0x0000_0003	bit0 ENABLE, bit1 RS, bit2 ABORT.
0x100	STBY_CR_EXTCAP_HEADER	R	0x0000_0012	Standby-CR capability header (CAP_ID 0x12).
0x104	STBY_CR_CONTROL	RW	0x0000_0000	Standby-CR control (bit5 self-clearing).
0x108	STBY_CR_DEVICE_ADDR	RW	0x0000_0000	Standby-CR device address.
0x10C	STBY_CR_CAPABILITIES	R	(cfg)	Standby-CR capabilities.
0x300..0x3FC	DAT window	RW	0x0	32 entries × 2 DWORDs (direct-access window).

Offset	Name	SW	Reset	Description
0x400..0x4FC	DCT window	RW	0x0	16 entries × 4 DWORDs (direct-access window).

Unimplemented offsets inside the window are **RAZ/WI**. Accesses outside the `NUM_I3C × 0x500` aperture → `TLM_ADDRESS_ERROR_RESPONSE`.

4. HC_CONTROL (0x04)

Bit	Name	Access	Notes
31	BUS_ENABLE	RW	1 = controller may drive transactions.
30	RESUME	RW (self-clear)	Resume from suspend; reads 0.
29	ABORT	RW (self-clear)	Flush pending commands; sets <code>TRANSFER_ABORT_STAT</code> ; reads 0.
12	HALT_ON_CMD_SEQ_TIMEOUT	RW	Halt vs continue on timeout.
8	HOT_JOIN_CTRL	RW	ACK/NACK hot-join.
7	I2C_DEV_PRESENT	RW	Legacy I2C devices present.
6	MODE_SELECTOR	R	Reads 1 (PIO mode).
0	IBA_INCLUDE	RW	Include broadcast address (OCA always includes).

Other bits are reserved (RAZ/WI).

5. RESET_CONTROL (0x10)

All bits are **write-1, self-clearing** (read back as 0):

Bit	Name	Effect (model)
0	SOFT_RST	Clears all queues + INTR/PIO_INTR status; CSR config preserved .
1	CMD_QUEUE_RST	Clears the command queue.
2	RESP_QUEUE_RST	Clears the response queue.
3	TX_FIFO_RST	Clears the TX data FIFO.
4	RX_FIFO_RST	Clears the RX data FIFO.
5	IBI_QUEUE_RST	Clears the IBI queue.

6. Interrupts

6.1 PIO_INTR_STATUS (0xA0)

Bit	Name	Type	Condition
0	TX_THLD_STAT	level	TX free entries $\geq 2^{(TX_BUF+1)}$.
1	RX_THLD_STAT	level	RX entries $\geq 2^{(RX_BUF+1)}$.
2	IBI_STATUS_THLD_STAT	level	IBI entries $\geq$ IBI_STATUS_THLD .
3	CMD_QUEUE_READY_STAT	level	Command free entries $\geq$ CMD_EMPTY_BUF_THLD .
4	RESP_READY_STAT	level	Response entries $\geq$ RESP_BUF_THLD .
5	TRANSFER_ABORT_STAT	latched W1C	Transfer aborted (HC_CONTROL.ABORT).
9	TRANSFER_ERR_STAT	latched W1C	NACK / TX underflow / RX overflow.

Level bits reflect current FIFO occupancy; writing 1 to a level bit has no persistent effect (it re-reflects the condition). Latched bits are cleared by write-1.

6.2 INTR_STATUS (0x20) — HC level (all latched, W1C)

Bit	Name
10	HC_INTERNAL_ERR_STAT
11	HC_SEQ_CANCEL_STAT
12	HC_WARN_CMD_SEQ_STALL_STAT (command-queue overflow)
13	HC_ERR_CMD_SEQ_TIMEOUT_STAT
14	SCHED_CMD_MISSED_TICK_STAT

INTR_FORCE (0x2C) write-1 sets the matching INTR_STATUS bit (test aid).

6.3 Aggregation

```
irq_o[i] = ((PIO_INTR_STATUS & PIO_INTR_SIGNAL_ENABLE) != 0)
          || ((INTR_STATUS      & INTR_SIGNAL_ENABLE)      != 0)
```

7. Command descriptor (model 64-bit layout)

The model command descriptor is a **firmware-facing functional abstraction** of the HCI transfer descriptor (it is intentionally simple and self-consistent; it is not bit-compatible with the raw HCI register-transfer descriptor):

Bits	Field	Meaning
[2:0]	CMD_ATTR	0 = regular transfer (only attr modelled).
[6:3]	TID	Transaction ID (echoed in the response).
[7]	RNW	0 = write, 1 = read.
[14:8]	DEV_INDEX	DAT index (0..31).
[15]	CP	1 = CCC transfer.
[39:32]	CCC_CODE	CCC command byte (when CP = 1).
[63:48]	DATA_LENGTH	Bytes to transfer.

The two `COMMAND_PORT` writes provide bits [31:0] then [63:32]; the second write enqueues the descriptor.

8. Response descriptor (32-bit) — `RESPONSE_PORT`

Bits	Field
[3:0]	TID (matches the command)
[27:16]	DATA_LENGTH (actual bytes transferred)
[31:28]	ERROR

ERROR codes: `0x0` SUCCESS, `0x1` CRC, `0x2` PARITY, `0x3` FRAME, `0x4` ADDRESS_NACK, `0x7` TRANSFER_ABORT, `0x8` I2C_W_NACK, `0x9` I2C_DATA_NACK, `0xF` OVERFLOW/UNDERFLOW.

9. IBI status descriptor (32-bit) — `IBI_PORT`

Bits	Field
[7:0]	IBI source address (addr < 1)
[15:8]	Payload length (bytes)

A read of `IBI_PORT` returns the status descriptor first, then successive reads return the payload DWORDs (little-endian, zero-padded to a DWORD boundary).

10. Transaction semantics (observable)

For a dequeued command with `HC_CONTROL.BUS_ENABLE = 1`:

1. The dynamic address is `DAT[DEV_INDEX].DWORD0[22:16]`.
2. **No bus model attached** $\Rightarrow$ `ERROR_ADDRESS_NACK`, `TRANSFER_ERR_STAT` set.
3. **Write** drains `ceil(DATA_LENGTH/4)` DWORDs from the TX FIFO; if insufficient $\Rightarrow$ `ERROR_OVERFLOW/UNDERFLOW`, `TRANSFER_ERR_STAT` set, no bus access.
4. The bus model is invoked; if it NACKs $\Rightarrow$ `ERROR_ADDRESS_NACK`.
5. **Read** pushes the returned bytes into the RX FIFO; if it would overflow $\Rightarrow$ `ERROR_OVERFLOW/UNDERFLOW`.

6. A response descriptor (error / actual length / TID) is pushed; if the response queue is full $\Rightarrow$ `HC_INTERNAL_ERR_STAT`.

The transaction completes `xfer_delay_ns` of simulated time after enqueue (a single configurable annotated latency, not waveform-accurate timing).

11. TLM-2.0 interface

- Only 32-bit, naturally-aligned accesses are accepted; others $\rightarrow$ `TLM_BURST_ERROR_RESPONSE`.
 - Byte-enables are rejected (`TLM_BYTE_ENABLE_ERROR_RESPONSE`).
 - Accesses $\geq$ aperture $\rightarrow$ `TLM_ADDRESS_ERROR_RESPONSE`.
 - Non-read/write commands $\rightarrow$ `TLM_COMMAND_ERROR_RESPONSE`.
 - `b_transport` annotates `access_delay_ns`.
 - `t_transport_dbg` provides side-effect-free back-door access to the DAT/DCT tables only (CSRs and FIFO ports are not exposed, to avoid perturbing queue state); malformed accesses return 0.
 - DMI is never granted (register/FIFO accesses have side effects).
-

12. Documented simplifications (vs RTL)

- Bit-level SCL/SDA signalling and OD/PP timing parameters are not modelled; `scl_o/sda_o/*_oe_o/sel_od_pp_o` are held idle.
- The `i3c_phy` CDC synchronisers are not modelled.
- Only the CSR-visible DAT/DCT direct-access windows are modelled (32/16 entries); the architectural 128-entry tables exported on the RTL `dat_mem/dct_mem` ports are abstracted away.
- HDR modes, the TCRI recovery datapath, and scan/DFT are not modelled; recovery outputs are held idle.